SIGMA-7 Emulator for Mac - November 2026.

Siggy is a MacOS emulator for an SDS Sigma 7. It allows interaction with a realistic panel display, a console, and terminal windows. There is also a visual debugger. It is written almost entirely in Swift (there are a few trivial C routines).

When I began this project, my goal was just to have something that could convey the look and feel of the machine. I had not coded anything significant on the Mac before and I wanted learn Swift anyway, so implementing the panel was a fun way to start. The CPU (i.e instruction emulator) was next. I compressed the IOP/DC/UNIT operations for devices into a simple IOP/Device model that satisfied the requirements of the Sigma IO instructions and does the actual I/O asynchronously.

A Sigma VM is represented on disc by a MacOS package file (i.e. directory) containing the necessary settings and data to represent a complete Sigma environment. It contains a sqlite3 database which contains tables for settings and the definition of virtual devices.

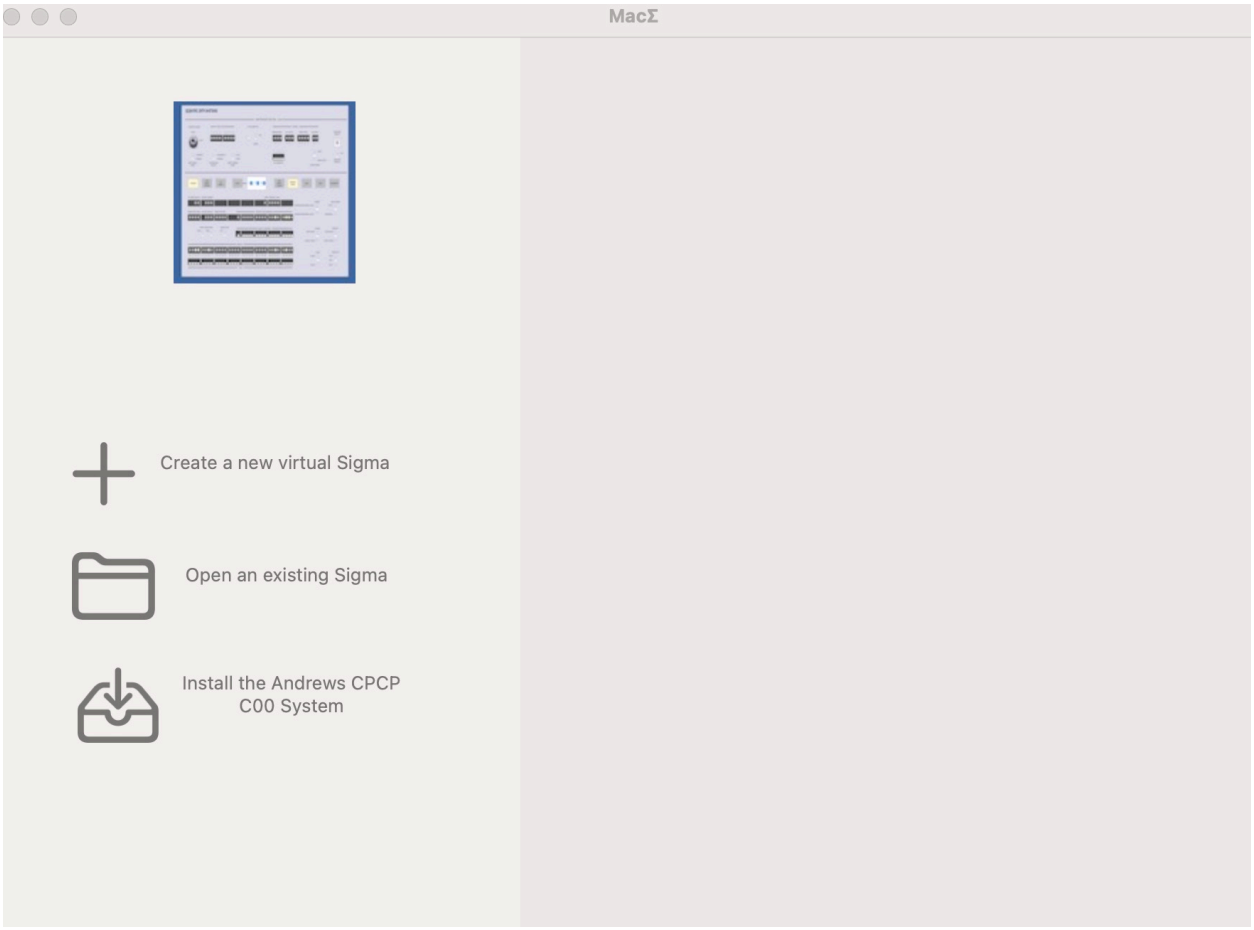
There are a variety of subtle differences between the various real Sigma CPU models. The initial emulator was for a minimal machine that would run CP-V: a basic Sigma-7 without decimal or float instructions, and the simpler memory map. The latest version is still a Sigma-7, but supports the large memory model for up to 4MB, as well as decimal and float instructions. Float instructions are necessary to run most fortran programs.

Installation

Install the Xcode source to an appropriate directory, and open the project with Xcode. You can build and run the emulator from within the Xcode IDE, however it will run considerably faster (about 16 times faster an M4 processor) if you build a deployable app using the Product/Archive menu. Once built, use the Distribute App/Custom/Copy functions to produce an app and move it to the Applications directory.

When first started, the emulator will create a sqlite3 database in your local library directory. This database contains tables for globally applicable (i.e. across all machines) settings, and a table of recently opened machines. It will also create a directory called MacΣ.logs in the user library directory for runtime log messages.

After initialization, a startup window is displayed, which provides the following functions:



1. Create a new virtual Sigma
   - Asks for a location to create the Virtual Sigma directory,
   - Asks for the location of a PO tape. This will get copied into the destination directory, Initially, I used a downloaded copy of Ken Rector's F00 SIMH PO, available here: https://github.com/kenrector/sigma-cpv-kit/blob/main/f00/f00disc/f00disc.tap. After a while I did a few SYSGENs and settled on a large memory configuration with more tapes, disk and a card reader and card punch. The card punch can be used to transfer files from the Sigma to the Mac host computer (and vice versa with the card reader - this is discussed later). You can select a zip file for the PO tape - If it contains a .mt or .tap file with the same name, it will be unzipped as it is copied to the target directory.
   - Creates a basic virtual hardware configuration including CPU, 8 IOPs, 16 discs, 4 RADs, 4 tapes, console, printer, card reader, card punch and a COC.
   - Allows the hardware configuration to be inspected and changed before booting.
   - Powers on the CPU and primes it for a tape boot.

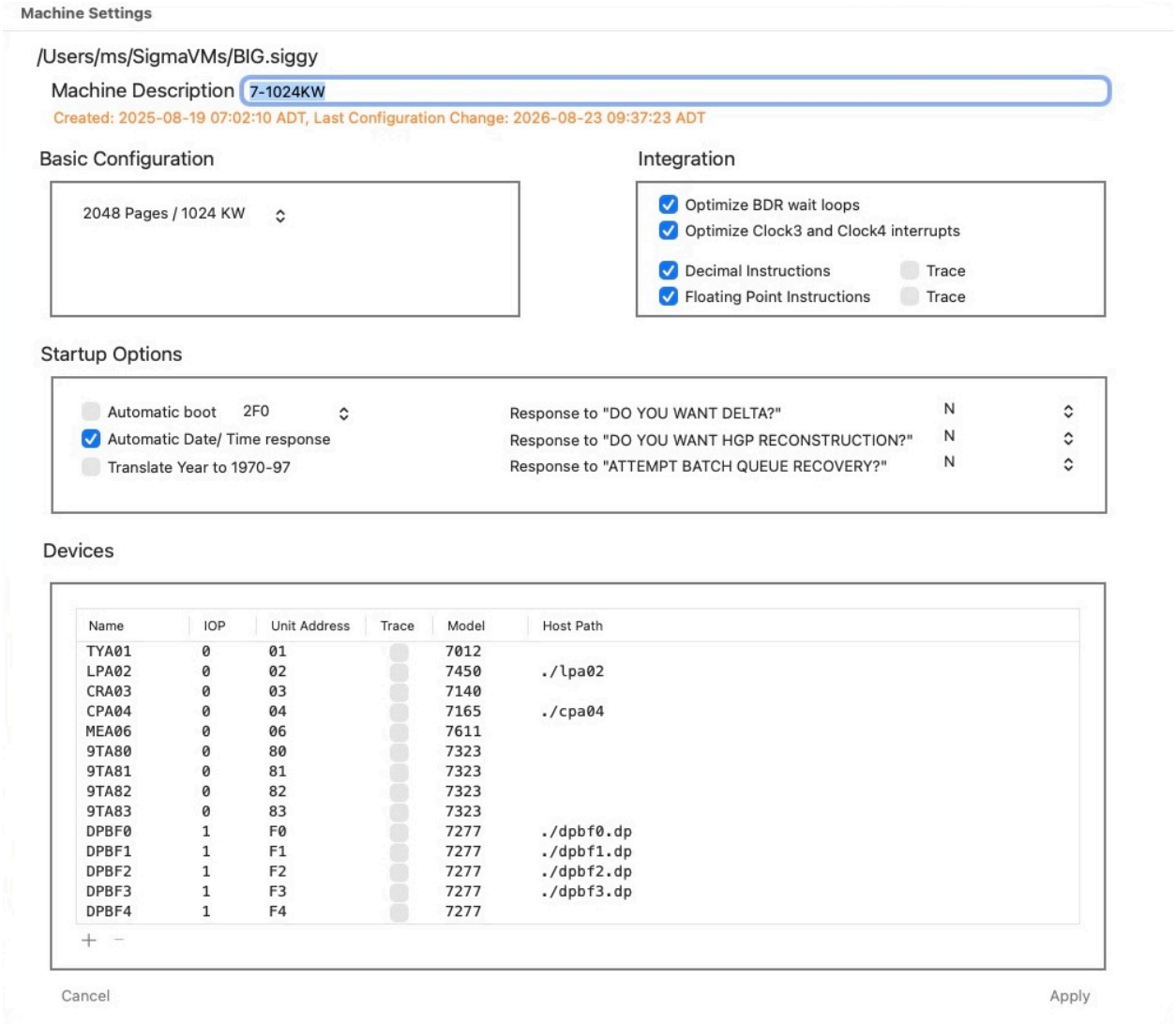
2. Open an existing Sigma
   - Opens a browser window to locate and open the virtual sigma of interest,
   - Powers on the selected sigma and primes it for a boot from the most recently used device.
   - If there are existing Sigma VMs that have been opened previously, they will appear in the right hand pane of the startup window. They can be started with a double click. They can also be removed from the recently opened list without affecting the underlying VM.

3. Install the Andrews CPCP C00 System
   - Asks for the location of the new virtual Sigma directory,
   - Creates a basic virtual hardware configuration,
   - Copies the DFBF0 and DPBF1 disk images,
   - Powers on the virtual Sigma and primes it for a disc boot.

Configuration

When creating a new Sigma, the configuration window is populated with a basic configuration and displayed.

The intention is that changes be made before the system is started. The configuration window can be accessed later from the menu, however if you modify the hardware configuration once the machine has started executing, it



will be reset.

Configuration parameters of note are:

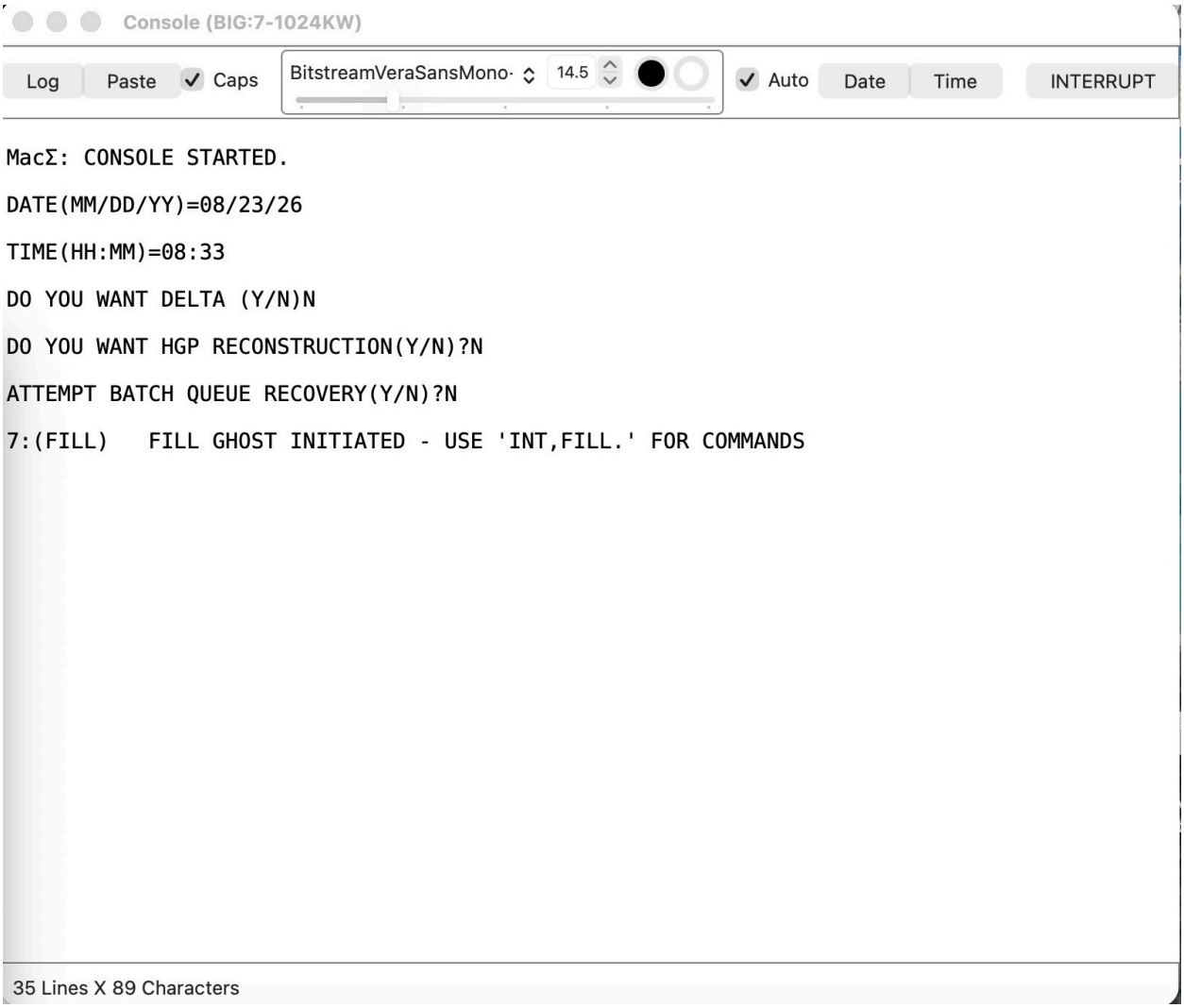
1. Memory: CP-V supports a maximum of 2048 pages (4MB). Sysgen will accept larger values for the CORE parameter, but the monitor is unable to handle this.
2. Optimize BDR wait loops detects BDR instructions that branch to themselves and calculates an approximate wall clock time to wait for.
3. Optimize CLOCK3 and CLOCK4 interrupts. When this is set, the interrupt is emulated instruction by instruction the first time it occurs, and thereafter the entire interrupt action (which is a MTW instruction) is emulated at once without going through the interrupt entry and exit sequences.
4. Automatic date time response is made by the console if selected (see below).
5. Individual devices can be examined and changed by double clicking on them. Devices.can be deleted and added from this window.

Click Apply when configuration is complete.

Console and Panel

The Console and Panel windows are displayed after configuration completes. These are the first windows displayed when an existing sigma is started. (An existing sigma can also be started from the desktop) by double clicking on its icon.

The console window is straightforward. The Date and Time buttons can be used to produce date and time responses to the boot time prompts. If “Translate Year” has been selected in the configuration, then the date is selected to be the current month and day, but a year between 1970 and 1997 that has the correct day of week and leap year characteristics as the current year.



The log button prompts for a Mac host file for logging console input and output.

The paste button causes the paste window to be displayed. This can be used to save / edit / recall prepared command lines. The paste buffers are saved in the virtual machine database, so they persist from one session to the next.

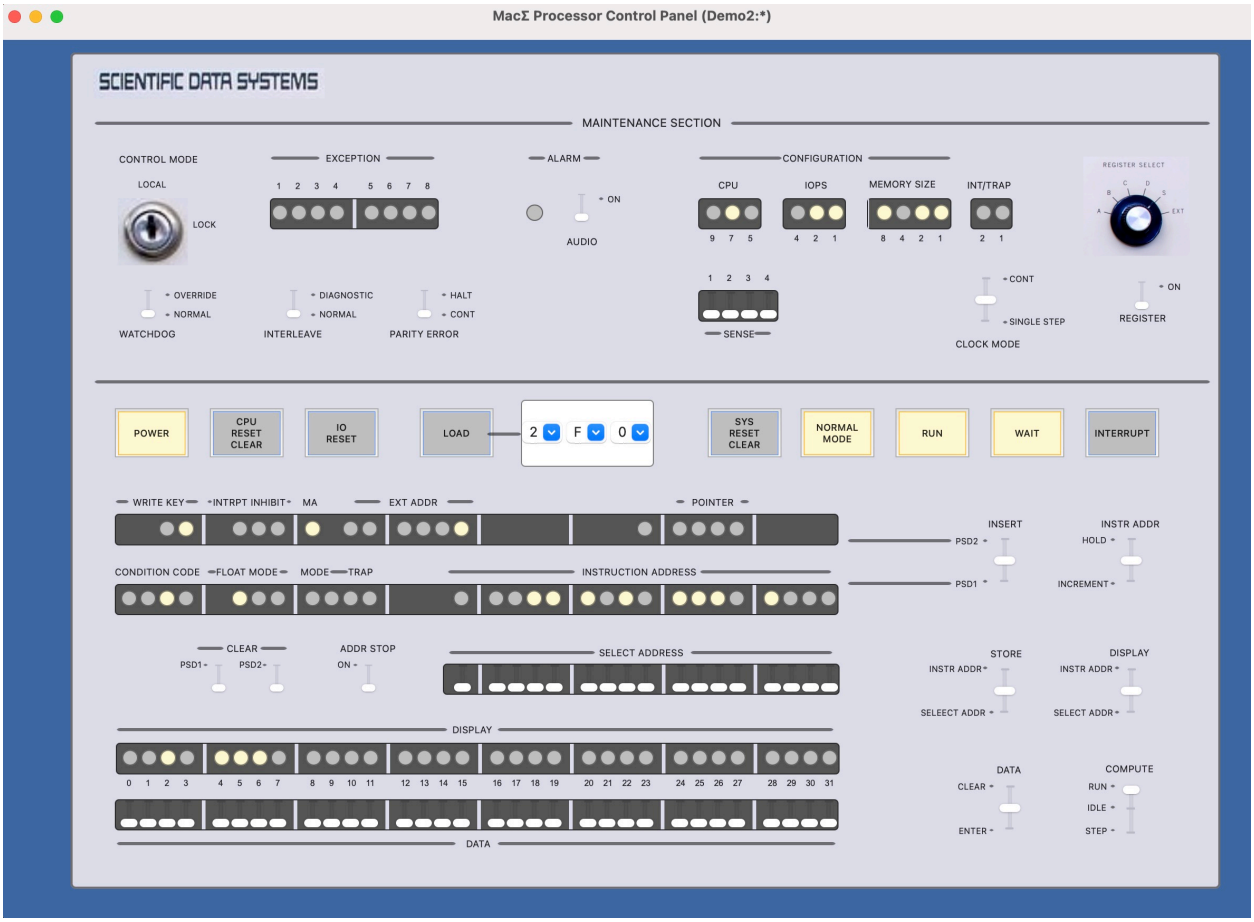
There is a selector for font size and foreground / background colours.

The auto button can be used to suppress (by unchecking it) the configured automatic responses to date/time/HGP recon/ etc prompts.

The panel window consists of various buttons, switches and lights. Some non-functional visual elements such as the key lock and the register select box (which should be a dial) are there to make things look right. In the upper maintenance section, the INT/TRAP lights work, but others have been repurposed (e.g. the memory fault lights show the recent trap or interrupt location), or don't work at all (ALARM). There is no reason why the alarm could not be implemented at some point. Most switches (other than the sense switches and the compute switch are not functional.

The lights in the lower section are updated about 5 times a second and produce an effect that is close to the way a real panel looked when the machine is running.

The Normal Mode button is red if a CPU fault occurs.



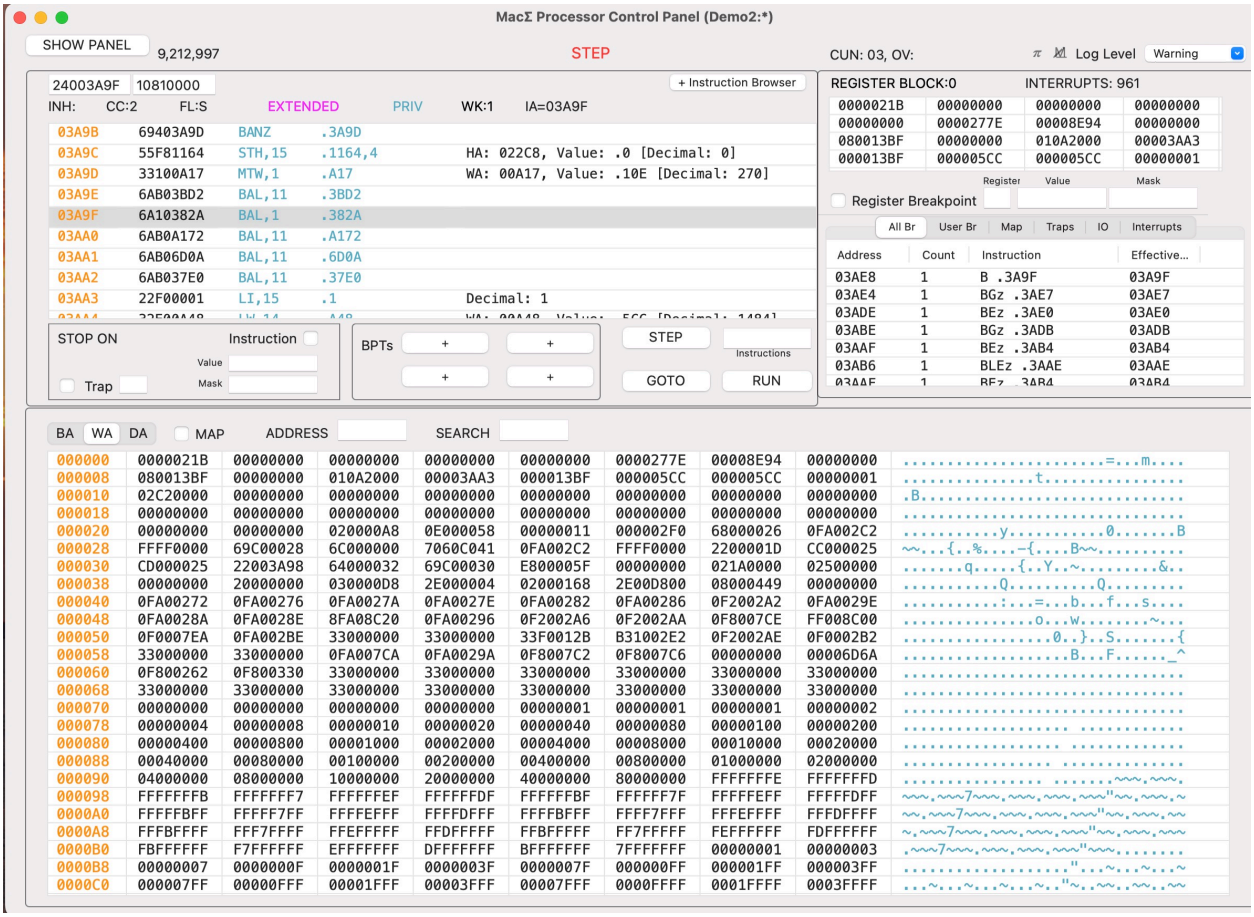
To start the machine, select the correct boot address and either push the Compute switch to run, or click the Run button.

For the initial tape, boot respond SFTP to the prompt for options. Use the date and time buttons on the console window to respond to the date and time prompts.

Debug Window

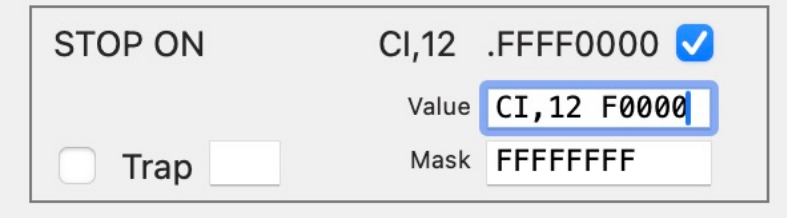
Press the Interrupt button to switch the from the panel display to the debugger. You can also use the View menu.

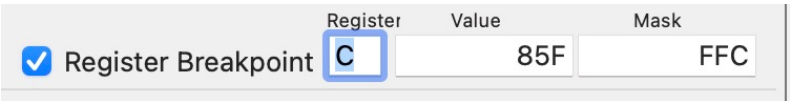
Press the Stop button in the debugger to pause the machine. Some controls in the debugger are not operative when the machine is running.



There is a lot to the debugger window. Starting at the top left, the Show-Panel button returns to the panel display. To its right is the instruction counter. The current execution status (in this case instruction step mode) is shown in red. On the right top side is the log level selector (various settings produce less or more log output). These log messages can be seen in the XCODE IDE and they are written to disk in the user library MacΣ.logs directory.

Below the status bar is an instruction display with the PSD in the top left. The current instruction is highlighted. The previous and next 32(or so) instructions can be scrolled. There are four breakpoints that can be set with the buttons in the center below the instruction window. Each breakpoint can stop on any combination of execute, read, write, or "transition" (execution transitions from below to above the specified address). To the left of the breakpoint section, there are selectable STOPs such as on a specific trap, a specific instruction opcode, and, in the register section, there is a single register breakpoint to stop when a specific register contains a specific value.



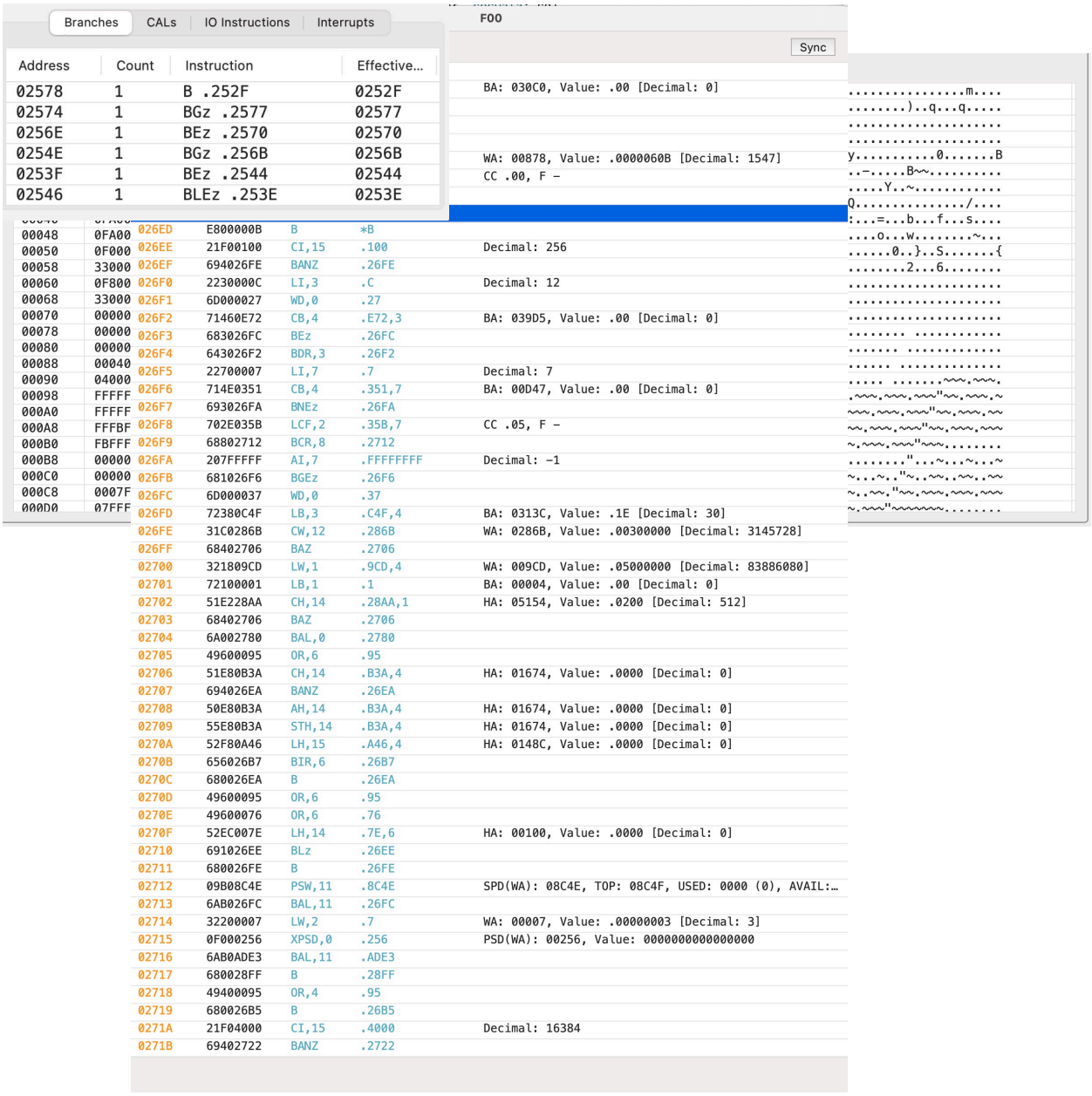


Above and to the right of the instruction display area the current user(CUN) and monitor overlay(OV) are displayed. The addresses of the S:CUN location and the P:NAME location are derived at the time the first mapped instruction is executed.

The GOTO button is useful when stepping through code and you want to skip several instructions or skip over a subroutine. Select the instruction in the display where you want to go and click goto. You can step multiple instructions at once by putting a number on the field to the right of the step button before it is pressed.

The + Instruction Browser button creates a new floating instruction browser window that is capable of scrolling all of memory
You can start multiple instruction browser windows if there is more than one section of code that needs to be observed. The sync button causes the instruction browser to be synchronized with the instruction display in the main debug window.

Below the register display you can select the last 32 branches (all and user mode); Map related intructions; Traps; IO instructions, and Interrupts.

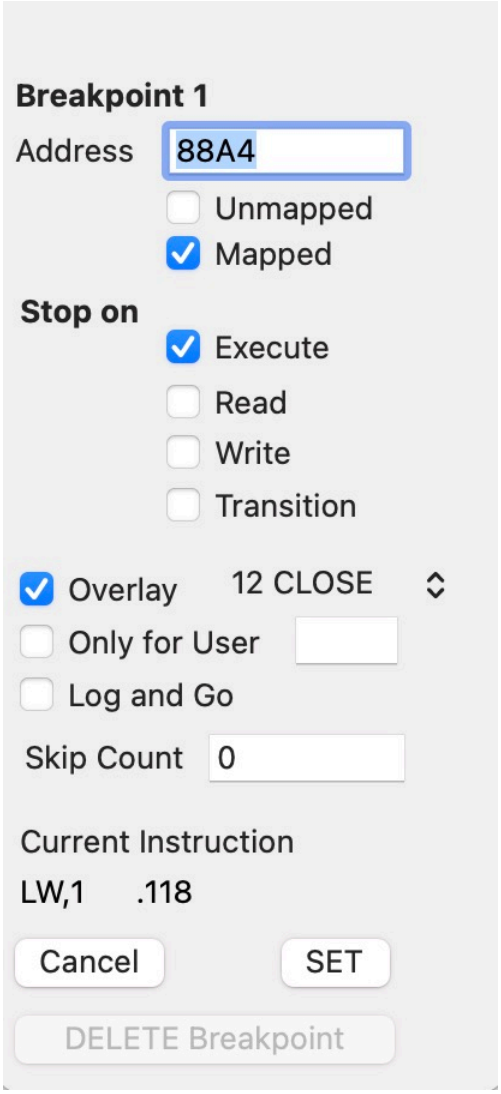


At the bottom is the memory display. You can scroll the entire contents of memory here. The BA/WA/DA selectors alter the address column on the left of the display. Check the MAP box to display the memory mapped (if possible). Entering a value in the address field causes the memory display to position to that address. The search field can be used to search for single word hex values or for text strings.

Memory can be altered from this window by entering word values in hex.

Breakpoints

To add a breakpoint, click on one of the four breakpoint buttons displaying a +. Various combinations are possible. A Transition breakpoint can catch a piece of code that moves from below a specific boundary address to something higher that the boundary. This is useful for catching jobs just starting.



Breakpoints can be specific to a user, and/or specific to a particular monitor overlay. Checking “Log and go” causes a a register dump to be written to the log file when the breakpoint is hit, and to be automatically continued.

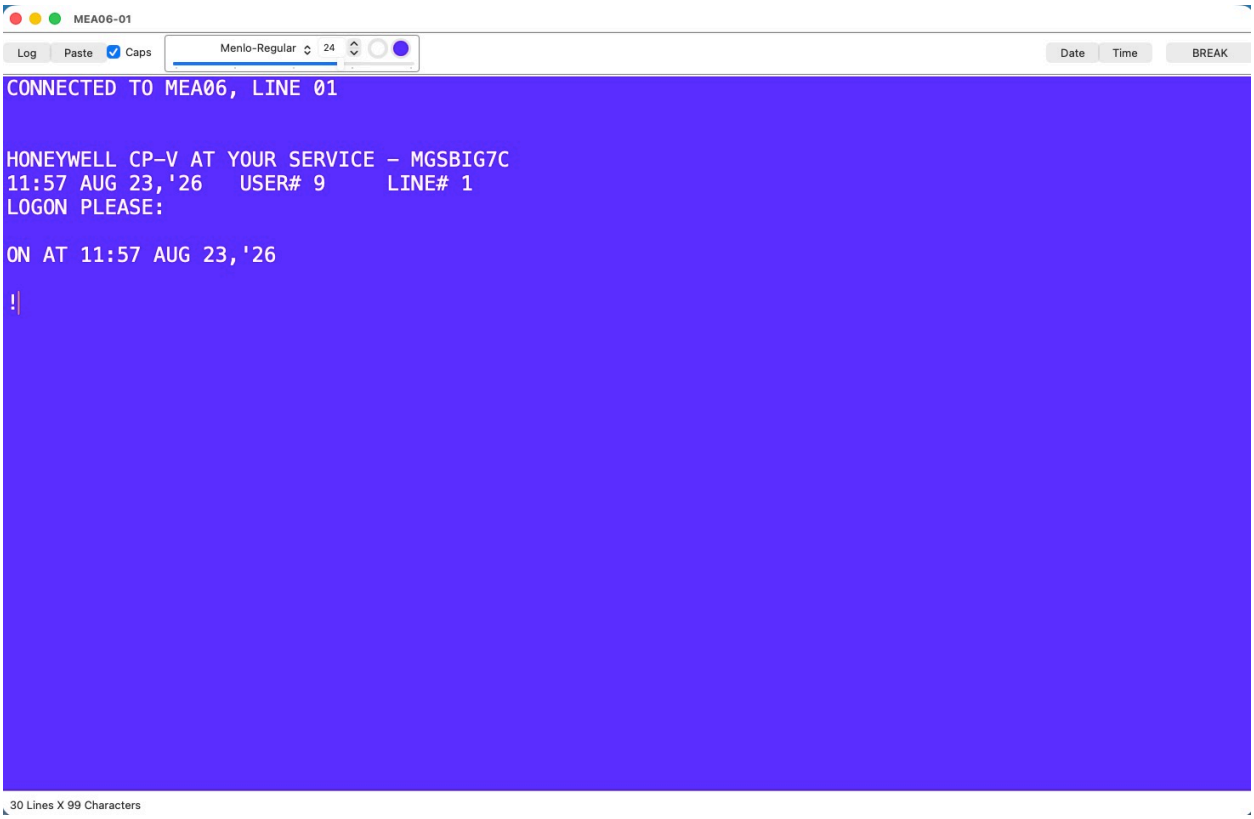
For F00, the tape boot and GENMDs take about 2.5 minutes to complete. There is a 6 minute video of the initial creation of a virtual machine and the tape boot here: https://drive.google.com/drive/folders/1UJpr-liFwXcDYjULk-btoQpfcFikM8nZ?usp=share_link

After the GENMDs complete, the system can be ZAPped and subsequently booted from the RAD. A RAD boot takes on the order of 3 seconds on a Mac with M1 processor.

Once FILL starts, it should be possible to logon.

Start a new terminal from the menu: click Machine > New Terminal Window. The terminal window is identical to the console window except that the “interrupt” button is replaced by the “break” button, and that there is no “auto” checkbox.

Use the :SYS,LBE account to logon initially.

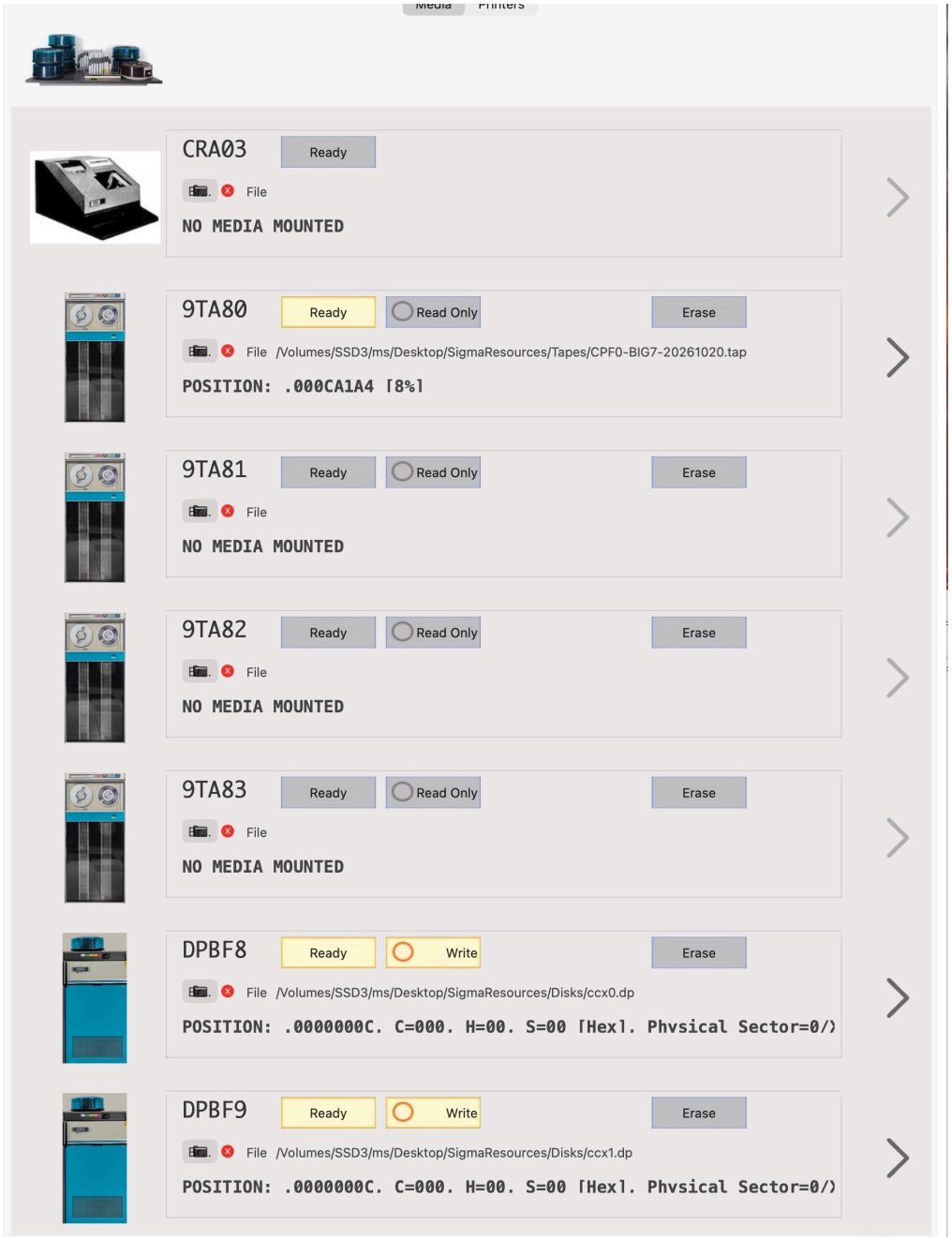


Both the terminal and the console can be scrolled back through the session history.

They are also capable of previous command editing: press the up arrow to show the last command line entered. Continue to press the up arrow to move backwards through the command line history. Once the command to be modified or re-issued is reached, the command can be changed by using the left arrow, right arrow to position the cursor and the tab key to toggle insertion mode. Press return to enter the edited line.

Media and Line Printer Options

This window is started by selecting Media and Printing from the Machine menu.

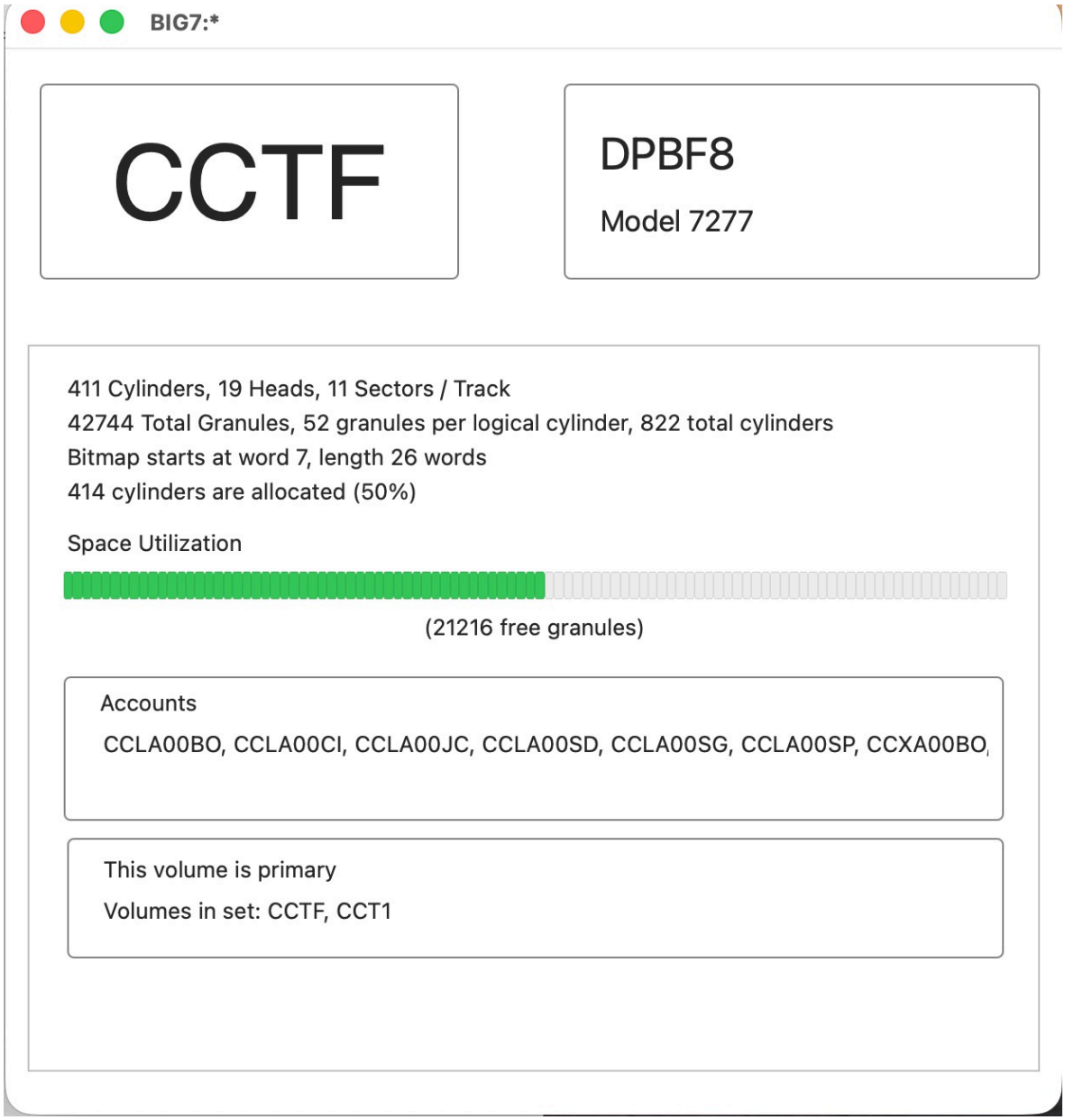


The media tab is used to connect Mac host files with CP-V devices. It is used when a tape or disk mount is requested on the console to connect the Mac file

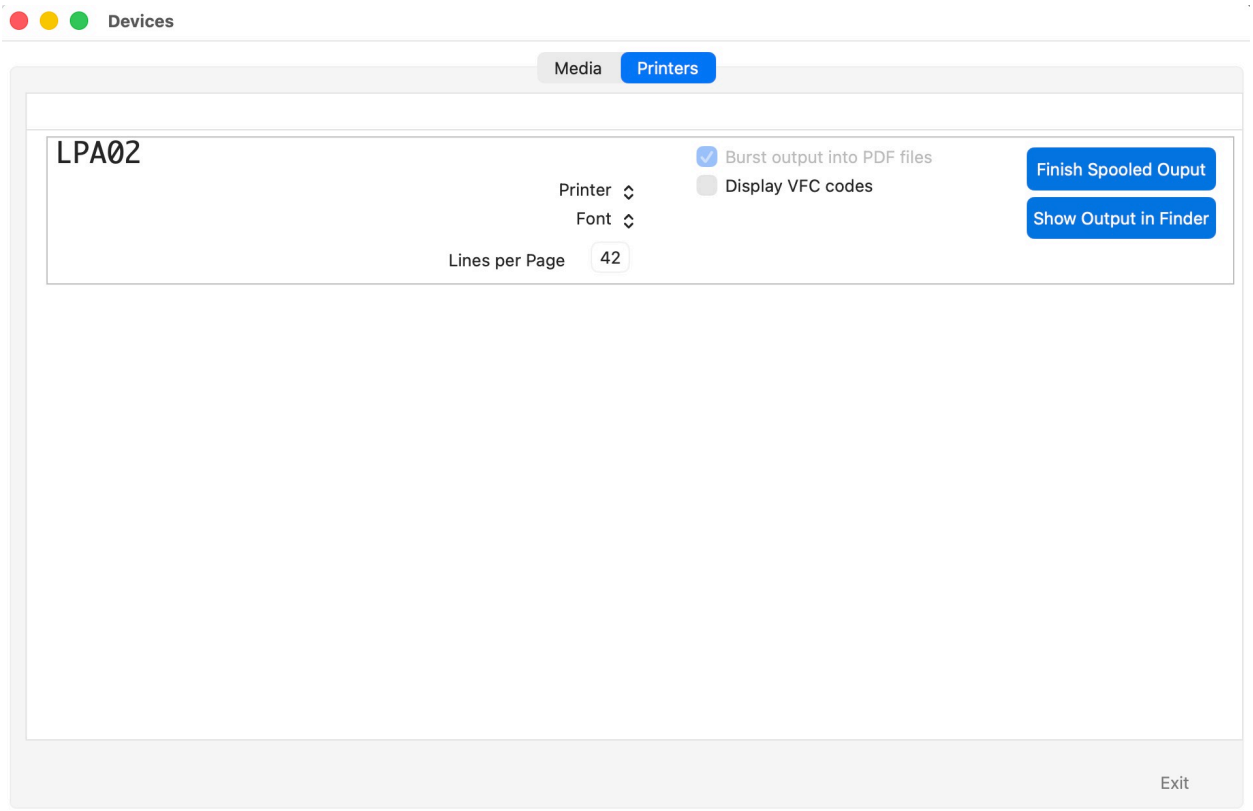
to the appropriate device. This can be done using the browse button in the entry for the appropriate device, or by dragging and dropping the applicable file.

Similarly for the card reader, a Mac file can be attached by dragging it to the Card reader entry in this window. Also, a Mac file that is dragged and dropped into the Panel window is also processed as a card reader input. Both tape and card handling work best with the HOTAVR and HOTCARD PASS2 options specified.

From the media window, click on the > symbol to see information about mounted media. The display for disk packs includes a measure of the space utilization and provides information about what account directories are on the device.

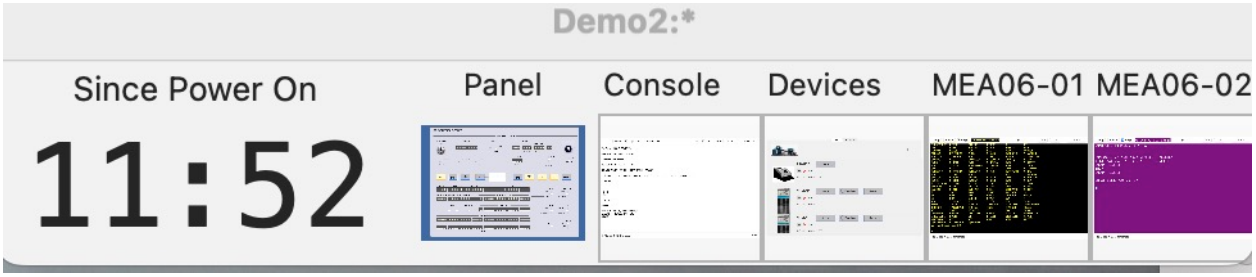


The printer tab can be used to change the lines per page, paper size and font for each printer device.



The Window Toolbar

The window toolbar is enabled and disabled from the view menu. It contains a session timer and buttons for each of the windows that belong to the virtual machine. Clicking one one of the buttons hides/shows the applicable window.



This is especially helpful on smaller displays.

Print and Punch Output

The View menu also contains options for displaying (using the Mac finder) the Print and Punch output directories. A job's printer output is typically collected and output to a pdf file. These files are grouped by logon in the virtual machine print directory.

Card punch output is also grouped by logon in the virtual machine punch directory. Each file is a txt file. Using PCL to copy an edit file to the card punch provides an easy way to convert the file to ascii and make it usable on the Mac.

Snapshots

Snapshots take a copy of the device files in the virtual machine directory. This is a considerably faster method of taking a backup than a complete system FSAVE. The snapshots do not include memory or other run-state information. They can be restored to a different virtual machine than the one from which they were taken.

